

PARÁMETROS GENÉRICOS

Se puede usar cualquier letra como nombre para los parámetros genéricos (como A, B, C, D, E, F, G, etc.) en lugar de los nombres tradicionales como T, U, V, S, siempre y cuando sigas la sintaxis correcta de los genéricos en Java. Los nombres de los parámetros de tipo en genéricos son **convenciones**, pero no están limitados a esas letras específicas. Puedes elegir cualquier letra o nombre que tenga sentido para ti o tu proyecto.

Ejemplo usando letras del abecedario (A, B, C, D, etc.)

A continuación, te muestro el mismo ejemplo anterior, pero usando letras del abecedario como parámetros de tipo en lugar de S, T, U, V, etc.:

```
import java.util.function.Function;

public class MultiComparator<A, B, C, D, E, F, G> {

    // Almacenamos las funciones de comparación
    private final Function<A, B> firstComparator;
    private final Function<B, C> secondComparator;
    private final Function<C, D> thirdComparator;
    private final Function<D, E> fourthComparator;
    private final Function<E, F> fifthComparator;
    private final Function<F, G> sixthComparator;
    private final Function<G, A> seventhComparator;

    // Constructor que acepta múltiples funciones de comparación
    public MultiComparator(Function<A, B> firstComparator,
        Function<B, C> secondComparator,
        Function<C, D> thirdComparator,
        Function<D, E> fourthComparator,
        Function<E, F> fifthComparator,
        Function<F, G> sixthComparator,
        Function<G, A> seventhComparator) {
        this.firstComparator = firstComparator;
        this.secondComparator = secondComparator;
        this.thirdComparator = thirdComparator;
        this.fourthComparator = fourthComparator;
        this.fifthComparator = fifthComparator;
        this.sixthComparator = sixthComparator;
        this.seventhComparator = seventhComparator;
    }

    // Método para aplicar todas las funciones de comparación
    public A applyAll(A input) {
        B result1 = firstComparator.apply(input);
        C result2 = secondComparator.apply(result1);
        D result3 = thirdComparator.apply(result2);
        E result4 = fourthComparator.apply(result3);
        F result5 = fifthComparator.apply(result4);
        G result6 = sixthComparator.apply(result5);
        return seventhComparator.apply(result6);
    }
}
```

```
public static void main(String[] args) {
    // Crear una instancia de MultiComparator con 7 funciones diferentes
    MultiComparator<Integer, String, Integer, Double, String, Long, Integer> comparator = new
MultiComparator<>()
    // Función 1: Convierte Integer a String
    input -> "Valor: " + input,
    // Función 2: Convierte String a Integer (número de caracteres)
    str -> str.length(),
    // Función 3: Convierte Integer a Double (multiplica por 2.5)
    length -> length * 2.5,
    // Función 4: Convierte Double a String
    result -> "Resultado: " + result,
    // Función 5: Convierte String a Long (longitud de nuevo string)
    result -> (long) result.length(),
    // Función 6: Convierte Long a Integer
    longResult -> longResult.intValue(),
    // Función 7: Devuelve Integer sumando 100 al resultado
    intResult -> intResult + 100
    );

    // Aplicar todas las funciones de comparación a un valor inicial
    Integer finalResult = comparator.applyAll(123);
    System.out.println("Resultado final: " + finalResult);
}
}
```

Explicación del código:

- En este código, en lugar de usar S, T, U, V, se usan las letras A, B, C, D, E, F, G como nombres de los parámetros de tipo.
- La lógica sigue siendo la misma: los tipos A, B, C, D, E, F, G son placeholders que representan tipos de datos genéricos que se pueden transformar de uno a otro usando las funciones definidas.

¿PUEDO USAR "N" PARA DATOS QUE NO SEAN NUMÉRICOS?

Sí, puedes usar "N" como un **parámetro genérico** para tipos de datos que **no sean numéricos**, pero es importante tener en cuenta que el uso de la letra "N" es solo una **convención** en la programación para indicar que se espera que el tipo sea un **número** (como **Number, Integer, Double**, etc.). Sin embargo, **Java no impone ninguna restricción** sobre qué tipo de datos puede asignarse a un parámetro genérico llamado "N". Puedes utilizar "N" para cualquier tipo de dato, ya sea numérico o no.

¿Qué son las convenciones de nombres en genéricos?

Las letras como **T, E, K, V, N**, etc., son **convenciones comunes** que se usan para mejorar la legibilidad del código. No son reglas obligatorias, y **Java no impone restricciones** sobre lo que representan estas letras.

Convenciones comunes:

- **T**: Tipo genérico.
- **E**: Elemento (utilizado en colecciones como List, Set).
- **K**: Llave (Key, usado en mapas como Map<K, V>).
- **V**: Valor (Value, usado en mapas como Map<K, V>).
- **N**: Número (usado a menudo para representar datos numéricos, pero no tiene que ser estrictamente numérico).

¿Puedo usar "N" para datos no numéricos?

Sí, puedes hacerlo. A pesar de que "N" es convencionalmente utilizado para representar **números**, Java te permite usarlo para cualquier tipo de dato.

Ejemplo usando "N" con tipos no numéricos:

A continuación, se muestra un ejemplo en el que usamos "N" como parámetro de tipo genérico para una clase que trabaja con **cadenas de texto (String)**, aunque convencionalmente "N" se usa para números:

```
public class GenericExample<N> {

    // Atributo de tipo genérico N
    private N value;

    // Constructor
    public GenericExample(N value) {
        this.value = value;
    }

    // Método para obtener el valor
    public N getValue() {
        return value;
    }

    // Método para establecer el valor
    public void setValue(N value) {
        this.value = value;
    }

    public static void main(String[] args) {
        // Crear una instancia de GenericExample con String (no
        numérico)
        GenericExample<String> stringExample = new
        GenericExample<>("Hola Mundo");

        // Mostrar el valor
        System.out.println("Valor: " + stringExample.getValue());

        // Crear una instancia de GenericExample con Integer
        (numérico)
        GenericExample<Integer> integerExample = new
        GenericExample<>(123);

        // Mostrar el valor
        System.out.println("Valor: " + integerExample.getValue());
    }
}
```

Explicación del código:

- Aquí, "N" se utiliza como un parámetro de tipo genérico, pero no hay restricciones sobre qué tipo puede ser.
- En el primer caso, el parámetro genérico "N" es **String** (no numérico), mientras que en el segundo caso es **Integer** (numérico).

Salida esperada:

Valor: Hola Mundo

Valor: 123

¿ENTONCES PUEDO USAR “Z” PARA DECLARAR UN TIPO DE DATO GENÉRICO EN LUGAR DE “T” Y PARA REPRESENTAR UNA KEY EN LUGAR DE “K” USAR LA LETRA “O”?

Sí, se puede usar cualquier letra, incluyendo "Z" para un tipo de dato genérico y "O" para representar una Key en lugar de "K". Java no impone restricciones sobre los nombres que uses para los parámetros de tipo genéricos, siempre y cuando sigas las reglas de sintaxis de los genéricos.

Uso de Z en lugar de T para tipos genéricos

Normalmente, se usa T como convención para representar un "tipo" genérico, pero esto es solo una convención. Puedes usar Z, X, Y o cualquier otra letra para hacer lo mismo.

Uso de O en lugar de K para representar una Key

De la misma manera, en colecciones como los mapas (Map<K, V>), es común usar K para "Key" y V para "Value", pero estas letras son convencionales. Puedes usar cualquier otra letra, como O para la clave y V para el valor.

```
import java.util.HashMap;
import java.util.Map;

public class CustomGenericExample<O, Z> {

    // Mapa genérico con clave de tipo O y valor de tipo Z
    private Map<O, Z> map;

    // Constructor
    public CustomGenericExample() {
        map = new HashMap<>();
    }

    // Método para agregar pares de clave-valor al mapa
    public void addItem(O key, Z value) {
        map.put(key, value);
    }

    // Método para obtener un valor dado una clave
    public Z getValue(O key) {
        return map.get(key);
    }

    // Método para mostrar el contenido del mapa
    public void printMap() {
        for (Map.Entry<O, Z> entry : map.entrySet()) {
            System.out.println("Clave (O): " + entry.getKey() + " - Valor
(Z): " + entry.getValue());
        }
    }
}
```

```
public static void main(String[] args) {
    // Crear una instancia del mapa usando String como clave y Integer
    como valor
    CustomGenericExample<String, Integer> example = new
    CustomGenericExample<>();

    // Agregar elementos al mapa
    example.addItem("Uno", 1);
    example.addItem("Dos", 2);
    example.addItem("Tres", 3);

    // Obtener un valor del mapa
    System.out.println("Valor asociado a 'Dos': " +
    example.getValue("Dos"));

    // Mostrar todo el contenido del mapa
    example.printMap();
}
}
```

Explicación del código:

- **O** y **Z** son los parámetros genéricos en la clase CustomGenericExample. **O** se usa para la **Key** (clave) y **Z** se usa para el **Value** (valor).
- Creamos un mapa (HashMap) donde **O** es el tipo de clave y **Z** es el tipo de valor.
- El ejemplo usa un mapa donde las claves son **String** (representadas por **O**) y los valores son **Integer** (representados por **Z**).

Salida esperada:

```
Valor asociado a 'Dos': 2
Clave (O): Tres - Valor (Z): 3
Clave (O): Uno - Valor (Z): 1
Clave (O): Dos - Valor (Z): 2
```

Conclusión:

- **Nombres convencionales vs. cualquier letra:** Aunque es convencional usar letras como T, U, V, etc., puedes usar cualquier letra o nombre que te resulte más intuitivo. Lo importante es que los nombres de los parámetros de tipo sean válidos y se mantenga la coherencia en el código.
- **Buena práctica:** Sin embargo, se recomienda seguir las **convenciones** en proyectos grandes o colaborativos para que otros desarrolladores puedan entender rápidamente el propósito de los parámetros genéricos. Pero en proyectos personales o casos más específicos, puedes usar cualquier letra o combinación de letras que prefieras.
- **Puedes usar "N" para cualquier tipo de dato** (numérico o no numérico), porque las letras en los genéricos son solo nombres de variables de tipo, no imponen restricciones en los tipos de datos que se les pueden asignar.